Nama : Aura Bilgenia Rachman

NIM : 264107020133

Kelas : TI-1C

JobSheet 8

Perulangan 2

Percobaan 1: Review Perulangan 1
Waktu Percobaan: 30 menit
Percobaan ini mengulang konsep perulangan FOR sebelum mempelajari perulangan bersarang. Program menampilkan karakter * sebanyak n kali secara mendatar (ke arah samping). Gunakan input n berupa bilangan bulat positif.

JobSheet08 > Star06.java > Star06

```java
import java.util.Scanner;

public class Star06 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Masukkan nilai n: ");
        int n = input.nextInt();

        for (int i = 1; i <= n; i++) {
            System.out.print("*");
        }
        System.out.println();
        input.close();
    }
}
```

Problems 12 Output Debug Consol

```
Masukkan nilai n: 5
*****
PS D:\DasPro\Project>
```

Pertanyaan

1. Jika inisialisasi i = 1 diubah menjadi i = 0 tanpa mengubah kondisi, berapa karakter * yang tercetak ketika n = 5? Jelaskan.
   - Jumlah karakter * yang tercetak: Sebanyak 6 karakter.
   - Penjelasan: Jika inisialisasi diubah menjadi int i = 0 dengan kondisi tetap i <= n dan $n = 5$, maka nilai i yang dieksekusi oleh perulangan adalah $0,1,2,3,4$, dan 5. Total ada 6 angka, sehingga perulangan mencetak 6 buah bintang (*).

2. Jika kondisi i <= n diganti menjadi i < n, berapa kali perulangan dijalankan saat n = 5? Mengapa hasilnya berbeda?
   - Jumlah perulangan: Dijalankan sebanyak 4 kali (untuk $n = 5$).
   - Penjelasan: Dengan inisialisasi $i = 1$ dan kondisi $i < n$, nilai i yang memenuhi syarat adalah $1,2,3$, dan 4 (karena saat $i = 5$, kondisi $5 < 5$ bernilai *false*). Hasilnya berbeda karena batas atas pengecekan lebih ketat (tidak menyertakan nilai n itu sendiri), sehingga perulangan berhenti satu langkah lebih awal.

3. Prediksi output potongan kode berikut ketika n = 9, kemudian buktikan dengan mengganti perulangan FOR pada program Anda dan menjalankannya.

   ```
   for (int i = 2; i <= n; i += 2) {
   System.out.print(i + " ");
   }
   ```

   Prediksi Output: 2 4 6 8

   Penjelasan: Perulangan dimulai dari $i = 2$ dengan penambahan i += 2 selama $i \leq 9$. Nilai i yang tercetak secara berurutan adalah:

   $i = 2$ (mencetak 2)

   $i = 4$ (mencetak 4)

   $i = 6$ (mencetak 6)

   $i = 8$ (mencetak 8)

   Saat $i = 10$, kondisi $10 \leq 9$ bernilai *false*, sehingga perulangan berhenti.

4. Modifikasi kode tersebut agar mencetak bilangan ganjil positif yang tidak melebihi n. Jelaskan mengapa perubahan increment i++ menjadi i-- (dengan kondisi tetap i <= n) membuat perulangan berjalan terus-menerus dan tidak berhenti dalam waktu wajar untuk n positif. Jangan menjalankan versi tersebut; jika terlanjur berjalan, hentikan program dengan menekan Ctrl + C pada terminal.

   ```
   for (int i = 1; i <= n; i += 2) {
      System.out.print(i + " ");
   }
   ```

   Jika increment i++ diubah menjadi i-- (decrement) dengan kondisi tetap i <= n untuk nilai n positif:

   - Perulangan dimulai dari $i = 1$ (yang memenuhi syarat $1 \leq n$).

- Alih-alih mendekati nilai n untuk memenuhi syarat berhenti, operasi i-- justru membuat nilai i semakin kecil (menjadi $0, -1, -2, -3$, dan seterusnya).
- Karena bilangan negatif dan nol selalu bernilai lebih kecil atau sama dengan angka positif n, kondisi i <= n akan selamanya bernilai *true*. Akibatnya, program terjebak dalam *infinite loop* (perulangan tanpa henti).

Percobaan 2: Pola Bintang Persegi Panjang (Nested FOR)
Waktu Percobaan: 30 menit
Pada percobaan ini digunakan nested loop FOR untuk mencetak persegi panjang bintang. Jumlah baris dan kolom dimasukkan secara terpisah sehingga peran kedua loop dapat diamati dengan jelas: outer loop mengatur baris, sedangkan inner loop mengatur kolom (ingat analogi rak/loker pada materi).
Contoh input baris = 3 dan kolom = 6 menghasilkan output:
******
******
******

JobSheet08 > Rectangle06.java > ...

```java
import java.util.Scanner;

public class Rectangle06 {
    Run | Debug
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Masukkan jumlah baris: ");
        int baris = input.nextInt();
        System.out.print("Masukkan jumlah kolom: ");
        int kolom = input.nextInt();

        for (int i = 1; i <= baris; i++) {
            for (int j = 1; j <= kolom; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
        input.close();
    }
}
```

```
Masukkan jumlah baris: 3
Masukkan jumlah kolom: 6
******
******
******
PS D:\DasPro\Project>
```

Pertanyaan

1. Tentukan fungsi variabel i dan j pada program tersebut.
   - Variabel i: Berfungsi sebagai counter (penghitung) untuk *outer loop* yang mengontrol jumlah baris yang dicetak secara vertikal (turun ke bawah).
   - Variabel j: Berfungsi sebagai counter (penghitung) untuk *inner loop* yang mengontrol jumlah kolom (karakter *) yang dicetak secara horizontal (ke samping) pada setiap barisnya.

2. Berapa kali outer loop, inner loop, dan perintah print("*") dijalankan untuk baris = 4 dan kolom = 7?
   - Eksekusi Outer Loop: Dijalankan sebanyak 4 kali (untuk nilai $i = 1,2,3,4$).
   - Eksekusi Inner Loop: Dijalankan sebanyak 28 kali secara keseluruhan (7 kali putaran untuk setiap 1 kali putaran outer loop, yaitu $4 \times 7 = 28$).
   - Eksekusi Perintah System.out.print("*"): Dijalankan sebanyak 28 kali, sehingga mencetak total 28 karakter bintang (*).

3. Apa yang terjadi jika batas perulangan luar dan dalam ditukar (outer menggunakan kolom, inner menggunakan baris)? Gunakan baris = 3 dan kolom = 6 untuk menjelaskan.

   ```
   Masukkan jumlah baris: 3
   Masukkan jumlah kolom: 6
   ***
   ***
   ***
   ***
   ***
   ***
   PS D:\DasPro\Project>
   ```

   Pola persegi panjang akan tertranspose (berubah orientasi). Program akan mencetak 6 baris ke bawah, di mana tiap barisnya berisi 3 karakter bintang (***), dari yang semula 3 baris berisi 6 bintang (******).

4. Jelaskan akibat penghapusan System.out.println(); setelah inner loop.
   Perintah System.out.println(); berfungsi untuk mencetak baris baru (*newline*) setelah satu baris bintang selesai dicetak. Jika perintah ini dihapus, semua bintang akan dicetak dalam satu baris lurus ke samping tanpa berpindah baris, sehingga tidak lagi membentuk persegi panjang melainkan satu baris panjang horizontal.

5. Modifikasi program agar karakter yang dicetak adalah #, lalu commit dan push perubahan ke GitHub

   ```
   Masukkan jumlah baris: 3
   Masukkan jumlah kolom: 6
   ######
   ######
   ######
   PS D:\DasPro\Project>
   ```

Percobaan 3: Debugging Bintang Segitiga – Nested WHILE

Waktu Percobaan: 45 menit

Kode awal pada percobaan ini sengaja dibuat belum menghasilkan format segitiga yang benar. Tugas Anda adalah mengidentifikasi kesalahannya dan memperbaikinya tanpa mengganti struktur WHILE menjadi FOR.

Output target untuk n = 5:

```
*
**
***
****
*****
```

JobSheet08 > Triangle06.java > ...

```java
import java.util.Scanner;

public class Triangle06 {
    Run | Debug
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Masukkan nilai n: ");
        int n = input.nextInt();

        int i = 0;
        while (i <= n) {
            int j = 0;
            while (j < i) {
                System.out.print("*");
                j++;
            }
            System.out.println();
            i++;
        }
        input.close();
    }
}
```

```
Masukkan nilai n: 5

*
**
***
****
*****
PS D:\DasPro\Project>
```

Pertanyaan

1. Sebutkan ketidaksesuaian output awal dan minimal dua perbaikan yang diperlukan untuk menghasilkan pola segitiga bertingkat.

   Ketidaksesuaian Output Awal: Karena inisialisasi dimulai dari $i = 0$, pada iterasi pertama saat $i = 0$, kondisi inner loop j < i (0 < 0) bernilai *false*, sehingga program langsung mencetak baris kosong (System.out.println()) sebelum mulai mencetak bintang pada baris berikutnya. Akibatnya, terdapat baris kosong di bagian paling atas pola segitiga.

   Minimal Dua Perbaikan yang Diperlukan:

   - Mengubah Inisialisasi i: Ubah nilai awal i dari int i = 0; menjadi int i = 1; agar perulangan baris langsung dimulai dari 1 dan mencetak bintang tanpa baris kosong di awal.
   - Menyesuaikan Kondisi Inner Loop / Outer Loop: Pastikan rentang nilai i dan j selaras dengan jumlah tingkat yang diinginkan pengguna (misalnya menggunakan kondisi j < i atau j <= i secara konsisten dengan batas nilai n).

2. Jelaskan peran variabel i dan j. Mengapa nilai j harus diinisialisasi kembali ke 0 pada setiap iterasi outer loop? Apa yang terjadi jika j tidak di-reset?

   - **Peran Variabel i:** Berfungsi sebagai pengontrol outer loop (perulangan luar) yang menentukan jumlah baris atau tingkat dari segitiga yang akan dicetak.
   - **Peran Variabel j:** Berfungsi sebagai pengontrol inner loop (perulangan dalam) yang menghitung jumlah karakter bintang (*) yang dicetak secara horizontal pada setiap baris.
   - **Mengapa j harus diinisialisasi kembali ke 0 (int j = 0;):** Agar setiap kali program berpindah ke baris baru (masuk ke iterasi outer loop berikutnya), penghitungan jumlah bintang selalu diatur ulang dari nol untuk baris tersebut.
   - **Apa yang terjadi jika j tidak di-reset (diletakkan di luar outer loop):** Nilai j tidak akan kembali ke 0 setelah baris pertama selesai, melainkan terus bertambah akumulatif. Akibatnya, pada baris-baris berikutnya nilai j sudah terlanjur lebih besar atau sama dengan i, sehingga kondisi j < i langsung bernilai false dan tidak ada bintang sama sekali yang tercetak pada baris-baris selanjutnya.

3. Jelaskan perubahan inisialisasi, kondisi, dan perubahan nilai (update) pada outer loop yang diperlukan untuk membentuk segitiga terbalik.

   - Inisialisasi: Diubah agar dimulai dari nilai maksimum, yaitu int i = n; (bukan dari 0 atau 1).
   - Kondisi: Diubah agar perulangan berjalan selama i > 0 (atau i >= 1), sehingga program terus berjalan hingga mencapai baris terakhir yang kecil.
   - Update (Perubahan Nilai): Diubah dari increment (i++) menjadi decrement (i--) agar pada setiap iterasi nilai i semakin berkurang (menyusut dari n turun ke 1).

Percobaan 4: Studi Kasus Nilai Tugas Proyek Kelompok

Waktu Percobaan: 75 menit

Dalam penilaian proyek kelompok, setiap kelompok memperoleh nilai dari sejumlah penilai. Program menerima jumlah kelompok dan jumlah penilai yang sama untuk setiap kelompok, kemudian menghitung rata-rata nilai tiap kelompok dan menentukan kelompok dengan ratarata tertinggi. Kasus ini diselesaikan dengan nested loop tanpa perlu menyimpan seluruh nilai: WHILE (luar) untuk kelompok, FOR (tengah) untuk penilai, dan DO-WHILE (dalam) untuk mengulang input nilai yang tidak valid.

JobSheet08 > NilaiKelompok06.java > ...

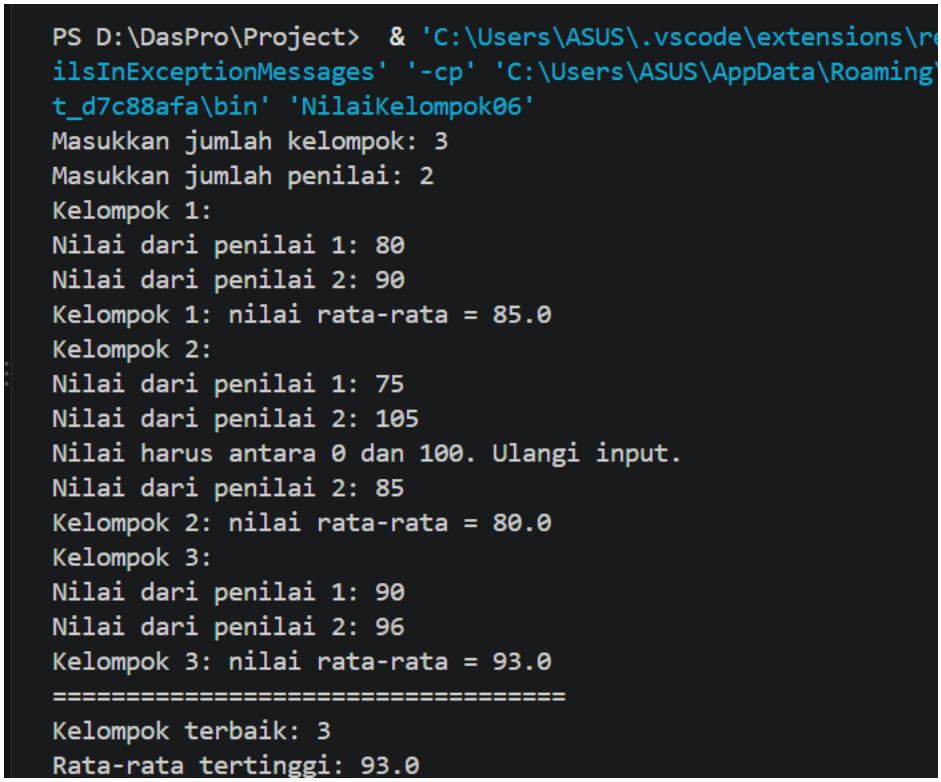
```java
import java.util.Scanner;

public class NilaiKelompok06 {
    Run | Debug
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Masukkan jumlah kelompok: ");
        int jumlahKelompok = input.nextInt();
        System.out.print("Masukkan jumlah penilai: ");
        int jumlahPenilai = input.nextInt();

        if (jumlahKelompok <= 0 || jumlahPenilai <= 0) {
            System.out.println("Jumlah kelompok dan penilai harus > 0.");
        } else {
            int nomorKelompok = 1;
            double rataTertinggi = -1;
            int kelompokTerbaik = 0;

            while (nomorKelompok <= jumlahKelompok) {
                System.out.println("Kelompok " + nomorKelompok + ":");
                double totalNilai = 0;
                for (int j = 1; j <= jumlahPenilai; j++) {
                    int nilai;
                    do {
                        System.out.print("Nilai dari penilai " + j + ": ");
                        nilai = input.nextInt();
                        if (nilai < 0 || nilai > 100) {
                            System.out.println("Nilai harus antara 0 dan 100. Ulangi input.");
                        }
                    } while (nilai < 0 || nilai > 100);
                    totalNilai += nilai;
                }
                double rataNilai = totalNilai / jumlahPenilai;
                System.out.println("Kelompok " + nomorKelompok + ": nilai rata-rata = " + rataNilai);
                if (rataNilai > rataTertinggi) {
                    rataTertinggi = rataNilai;
                    kelompokTerbaik = nomorKelompok;
                }
                nomorKelompok++;
            }
            System.out.println("==================================");
            System.out.println("Kelompok terbaik: " + kelompokTerbaik);
            System.out.println("Rata-rata tertinggi: " + rataTertinggi);
        }

        input.close();
    }
}
```



Pertanyaan

1. Mengapa totalNilai perlu di-reset di dalam outer loop, tetapi rataTertinggi tidak boleh di-reset pada setiap kelompok?
   - **totalNilai perlu di-reset (totalNilai = 0):** Karena totalNilai digunakan untuk menghitung akumulasi (jumlah) nilai dari para penilai untuk **satu kelompok saja**. Jika tidak di-reset ke 0 saat berpindah ke kelompok baru, nilai dari kelompok sebelumnya akan terbawa dan terjumlahkan ke kelompok berikutnya, sehingga rata-rata kelompok selanjutnya menjadi tidak akurat.
   - **rataTertinggi tidak boleh di-reset pada setiap kelompok:** Karena rataTertinggi bertindak sebagai variabel perekam nilai tertinggi global yang membandingkan rata-rata antar seluruh kelompok. Nilai ini harus mempertahankan nilai tertinggi yang pernah ditemukan dari kelompok-kelompok sebelumnya agar di akhir program kita dapat mengetahui kelompok mana yang meraih rata-rata tertinggi.

2. Untuk 3 kelompok dan 2 penilai (tanpa input yang salah), berapa kali badan perulangan FOR dijalankan secara keseluruhan?
   - Jumlah Eksekusi: Dijalankan sebanyak 6 kali secara keseluruhan.
   - Penjelasan:
     a. *Outer loop* (while) berjalan sebanyak 3 kali (karena ada 3 kelompok).
     b. Pada setiap iterasi *outer loop*, perulangan for berjalan sebanyak 2 kali (karena ada 2 penilai).
     c. Total eksekusi badan perulangan for = 3 kelompok $\times$ 2 penilai $=$ 6 kali.

3. Apa yang terjadi jika totalNilai dideklarasikan sebagai int dan rata-rata dihitung menggunakan pembagian dua bilangan bulat? Berikan contoh nilai.
   - Dalam Java, jika terjadi pembagian antara dua nilai bertipe integer (int / int), maka operasi yang terjadi adalah pembagian bulat (integer division). Bagian desimal/pencacah akan langsung diabaikan (dibuang/dibulatkan ke bawah ke bilangan bulat terdekat), sehingga hasil perhitungan rataNilai menjadi kurang presisi.
   - Contoh Nilai: Misalnya jumlahPenilai = 2. Penilai 1 memberi nilai 85 dan Penilai 2 memberi nilai 80.
     Total nilai = $85 + 80 = 165$.
     Secara matematis, rata-rata asli adalah $165/2 = 82.5$.
     Namun jika totalNilai bertipe int, hasil dari 165 / 2 menghasilkan 82 (bagian desimal .5 hilang), lalu diubah ke double menjadi 82.0.

4. Mengapa rataTertinggi diberi nilai awal -1, bukan 0? Berikan contoh input yang membuat hasil salah apabila nilai awalnya 0.
   Agar nilai rata-rata kelompok pertama yang diproses pasti lebih besar daripada rataTertinggi awal, sehingga kelompok pertama dapat langsung ditetapkan sebagai kelompok terbaik sementara. Selain itu, rentang nilai valid yang dimasukkan adalah $0 \leq$ nilai ≤ 100.
   Contoh Input yang Menyebabkan Hasil Salah Jika Nilai Awal 0:
   - Andaikan ada 2 kelompok dan nilai yang diberikan penilai untuk kedua kelompok semuanya adalah 0 (misalnya karena gagal atau diskualifikasi).
   - Rata-rata Kelompok 1 = 0.0, Rata-rata Kelompok 2 = 0.0.
   - Jika rataTertinggi diinisialisasi dengan 0:
     - Saat pengecekan if (rataNilai > rataTertinggi), kondisi 0.0 > 0.0 bernilai *false*.
     - Akibatnya, blok if tidak pernah dieksekusi. Nilai kelompokTerbaik akan tetap bernilai 0 (sesuai inisialisasi awal kelompokTerbaik = 0), yang berarti program gagal mencatat bahwa Kelompok 1 adalah kelompok terbaik yang ada.

5. Program ini memiliki tiga tingkat perulangan. Sebutkan masing-masing jenis perulangan beserta perannya.

- **while (Tingkat Utama / Outer Loop)** Mengontrol perulangan antar kelompok. Perulangan ini berjalan dari kelompok pertama hingga mencapai jumlahKelompok.
- **for (Tingkat Kedua / Middle Loop)** Mengontrol perulangan penilai untuk kelompok yang sedang aktif. Perulangan ini berjalan dari penilai ke-1 sampai jumlahPenilai.
- **do-while (Tingkat Ketiga / Inner Loop)** Melakukan validasi input nilai penilai. Perulangan ini memastikan input nilai tidak boleh < 0 atau > 100, dan akan terus meminta ulang input jika pengguna memasukkan nilai di luar rentang valid tersebut.

Tugas

1. **Tugas 1: Pola Bilangan Bertingkat**

Buat program untuk mencetak pola bilangan 1 sampai nomor baris. Masukkan bilangan bulat n (n minimal 1). Pada baris pertama tercetak 1, pada baris kedua 1 2, dan seterusnya sampai baris ke-n. Setelah pola selesai, tampilkan total banyaknya angka yang telah dicetak. Simpan dengan nama file: Tugas1NoAbsen.java.

Contoh untuk n = 5:

Masukkan nilai n = 5

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

Total angka tercetak: 15

JobSheet08 > Tugas1_06.java > ...

```java
import java.util.Scanner;

public class Tugas1_06 {
    Run | Debug
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Masukkan nilai n = ");
        int n = input.nextInt();

        int totalAngka = 0;

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print(j + " ");
                totalAngka++;
            }
            System.out.println();
        }
        System.out.println("Total angka tercetak: " + totalAngka);
        input.close();
    }
}
```

```
Masukkan nilai n = 5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
Total angka tercetak: 15
PS D:\DasPro\Project>
```

2. Tugas 2: Tabel Perkalian

Buat program untuk menampilkan tabel perkalian berukuran n × n menggunakan nested FOR atau WHILE. Nilai n minimal 3. Setiap posisi menampilkan hasil perkalian nomor baris dan nomor kolom. Atur spasi agar keluaran rapi, lalu tampilkan jumlah seluruh angka pada tabel. Simpan program dengan nama file: Tugas2NoAbsen.java.

Contoh untuk n = 4:

Masukkan nilai n = 4

1	2	3	4
2	4	6	8
3	6	9	12
4	8	12	16

Jumlah seluruh angka: 100

> Petunjuk: agar angka tersusun rata kanan, gunakan System.out.printf("%4d", hasil); yang mencetak bilangan dengan lebar 4 karakter, atau gunakan karakter tab "\t" sebagai pemisah.

JobSheet08 > Tugas2_06.java > ...

```java
import java.util.Scanner;

public class Tugas2_06 {
    Run | Debug
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Masukkan nilai n = ");
        int n = input.nextInt();

        int jumlahSeluruhAngka = 0;

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                int hasil = i * j;
                System.out.printf("%4d", hasil);
                jumlahSeluruhAngka += hasil;
            }
            System.out.println();
        }

        System.out.println("Jumlah seluruh angka: " + jumlahSeluruhAngka);

        input.close();
    }
}
```

```
Masukkan nilai n = 4
   1   2   3   4
   2   4   6   8
   3   6   9  12
   4   8  12  16
Jumlah seluruh angka: 100
PS D:\DasPro\Project>
```

3. Tugas 3: Rekap Penjualan

Beberapa Toko Sebuah pusat perbelanjaan memiliki beberapa toko. Setiap toko melayani sejumlah pelanggan pada hari yang sama, dan setiap pelanggan membeli beberapa barang. Program meminta jumlah toko, kemudian jumlah pelanggan pada masing-masing toko, dan jumlah barang yang dibeli setiap pelanggan. Hitung dan tampilkan total pelanggan serta total barang per toko, total barang seluruh toko, dan nomor toko dengan jumlah barang terbanyak. Jika terdapat nilai tertinggi yang sama, tampilkan toko dengan nomor paling kecil. Simpan program dengan nama file: Tugas3NoAbsen.java.

Ketentuan: jumlah toko minimal 1; jumlah pelanggan dan jumlah barang setiap pelanggan boleh 0, tetapi tidak boleh negatif. Gunakan perulangan bersarang; data tidak perlu disimpan satu per satu, cukup diakumulasikan ke variabel total.

Contoh: Toko 1 memiliki 2 pelanggan yang membeli 3 dan 2 barang; Toko 2 memiliki 3 pelanggan yang membeli 1, 4, dan 2 barang. Ringkasan yang diharapkan:
Toko 1 - pelanggan: 2, total barang: 5
Toko 2 - pelanggan: 3, total barang: 7
Total barang seluruh toko: 12 Toko dengan barang terbanyak: 2

JobSheet08 > Tugas3_06.java > ...

```java
import java.util.Scanner;

public class Tugas3_06 {
    Run | Debug
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Masukkan jumlah toko: ");
        int jumlahToko = input.nextInt();

        int totalBarangSeluruhToko = 0;
        int maxBarang = -1;
        int tokoTerbanyak = 0;

        for (int i = 1; i <= jumlahToko; i++) {
            System.out.print("Masukkan jumlah pelanggan untuk Toko " + i + ": ");
            int jumlahPelanggan = input.nextInt();

            int totalBarangToko = 0;

            for (int j = 1; j <= jumlahPelanggan; j++) {
                System.out.print("Masukkan jumlah barang yang dibeli pelanggan " + j + ": ");
                int jumlahBarang = input.nextInt();
                totalBarangToko += jumlahBarang;
            }

            totalBarangSeluruhToko += totalBarangToko;
            System.out.println("Toko " + i + " - pelanggan: " + jumlahPelanggan + ", total barang: " + totalBarangToko);

            if (totalBarangToko > maxBarang) {
                maxBarang = totalBarangToko;
                tokoTerbanyak = i;
            }
        }
        System.out.println("Total barang seluruh toko: " + totalBarangSeluruhToko);
        System.out.println("Toko dengan barang terbanyak: " + tokoTerbanyak);

        input.close();
    }
}
```

```
Masukkan jumlah toko: 2
Masukkan jumlah pelanggan untuk Toko 1: 2
Masukkan jumlah barang yang dibeli pelanggan 1: 3
Masukkan jumlah barang yang dibeli pelanggan 2: 2
Toko 1 - pelanggan: 2, total barang: 5
Masukkan jumlah pelanggan untuk Toko 2: 3
Masukkan jumlah barang yang dibeli pelanggan 1: 1
Masukkan jumlah barang yang dibeli pelanggan 2: 4
Masukkan jumlah barang yang dibeli pelanggan 3: 2
Toko 2 - pelanggan: 3, total barang: 7
Total barang seluruh toko: 12
Toko dengan barang terbanyak: 2
PS D:\DasPro\Project>
```